CineRoll — Reparto de responsabilidades, Fase 3

Plataforma de Cine, Series y Anime · Actualizado el 12 de noviembre de 2026

1. Dónde estamos

La Fase 2 (API del catálogo, admin y permisos) está cerrada. Revisando `main`, este es el estado real:

Área	Estado en main
Base del proyecto	Django + DRF + JWT + CORS, settings de local y de producción, PostgreSQL (o SQLite con `USE_SQLITE`), formato único de errores (`config/exceptions.py`), pruebas automáticas en cada PR. Listo.
Catálogo (media)	CRUD de `Media`, `Genero` y `Persona` con escritura solo para administradores; `/api/personas/`; búsqueda, filtros (género, tipo, año, estado, `?cine=`), ordenamiento por fecha, título y puntuación; `seed_data` con fichas, pósters y próximos estrenos. Listo.
Cartelera (cines)	Scraper de Cinemark (funciones y `UpcomingReleases`), normalización y matching de títulos, `TituloPendiente`, servicio `scrape_and_save_cartelera`, limpieza de funciones vencidas en cada ejecución, endpoints `/api/cines/`, `/api/cines/<id>/funciones/`, `/api/funciones/` y `/api/media/<id>/funciones/`. Listo para 1 cine.
Reconocimiento de cines	Cinépolis (elegido como segundo scraper), Royal Films y Cineland documentados con URL, parámetros y forma de la respuesta en `RECONOCIMIENTO_CARTELERA.md`. Los tres tienen sede en Valledupar.
Automatización	Celery y Redis instalados (`config/celery.py`, `docker-compose.yml`) con una tarea de prueba (`cines.tasks.ping`). Todavía no hay tareas reales ni horarios programados.
Usuarios y reseñas	Registro, login, refresh, logout con lista negra, cambio de contraseña, perfil propio y público, reseña propia, “mis reseñas”, escala de calificación, reseñas con promedio y moderación en el admin. Listo.
Frontend	Login, registro, perfil y cambio de contraseña conectados a la API real. Aún no hay pantallas de catálogo ni de cartelera.

Esto significa que lo que falta ahora es el corazón de la propuesta de valor: **que la cartelera real de Valledupar se cargue sola**. Hoy solo existe el scraper de Cinemark, y la cartelera de Cinemark en Valledupar estaba vacía, así que la base todavía no tiene funciones reales de la ciudad.

Al revisar el código encontré algunos detalles que conviene resolver dentro de esta fase (cada uno está asignado en las secciones de las personas):

- **El contrato del scraper está atado a Cinemark.** `CinemarkCartelera`, `CinemarkUpcomingRelease`, `_get_scraper` y `CINEMA_WEBSITES` viven en el código de Cinemark o solo conocen a Cinemark. Para sumar más cines hay que generalizarlos primero.
- **Las ciudades no coinciden entre cines.** Cinemark usa el slug `bogota`, Cinépolis usa `valledupar-colombia`, Royal Films usa el ID `1006` y Cineland el código `0051`. Sin una ciudad canónica (`valledupar`), `Cine.ciudad` quedaría con valores distintos y el filtro `?ciudad=` fallaría.
- **Las funciones fantasma.** Si una función cambia de sala o formato en la fuente, se guarda como una nueva y la anterior queda hasta que pase su hora. Tampoco se borran las funciones futuras que el cine retiró.
- **Casi toda la cartelera real quedará como título pendiente.** El catálogo actual tiene clásicos y algunos estrenos; los títulos que hoy están en cartelera (por ejemplo, los que ya vimos en Cinépolis y Cineland) no existen en `Media`, y no hay una forma ágil de darlos de alta.
- **Una prueba que nunca corre.** En `media/tests/test_seed_data.py`, la prueba `test_no_pisa_estado_ni_fecha_de_titulos_existentes` quedó indentada dentro de otra prueba, así que Django nunca la ejecuta.
- **Limpieza menor en el código:** imports duplicados (y `from html import parser`) en `scrape_cartelera.py` y en `test_scrape_command.py`; en el frontend, `API_BASE_URL` está fija en `localhost:8000`, y el registro y el cambio de contraseña dicen “mínimo 6 caracteres” cuando el código exige 8. Los botones de Google y Apple y el enlace “¿Olvidaste tu contraseña?” no hacen nada.

4. Objetivo de la Fase 3

Que la cartelera de los cines de Valledupar se cargue de forma automática, sin intervención manual: un scraper por cine, un formato común, sin duplicados ni funciones fantasma, y Celery Beat ejecutándolos una o dos veces al día. De paso se adelanta la capa social para llegar a la Fase 4 con los modelos ya construidos.

Reglas de trabajo: se mantienen las mismas del reparto anterior (una rama `feature/<area>-<descripcion>` por funcionalidad, PR obligatorio con al menos una revisión, commits pequeños, secretos solo en `.env`, migraciones solo de las apps propias y sincronización semanal de 15 a 20 minutos). Reglas nuevas por ser una fase de scraping:

- **Scraping responsable.** Solicitudes moderadas y espaciadas (pausa de al menos 1 segundo entre peticiones al mismo sitio), un `User-Agent` identificable y nunca más de dos ejecuciones al día por cine.
- **Las pruebas no usan la red.** Cada scraper se prueba con una respuesta de ejemplo guardada en un archivo de prueba (con los horarios recortados y sin datos de sesión de usuario), tal como ya se hace con Cinemark.
- **Los scrapers nuevos no tocan la lógica compartida.** Solo devuelven el formato común; guardar, relacionar títulos y limpiar sigue haciéndolo `cines/services.py`.

5. Persona 1 — Líder técnico e integración

Sigue siendo quien destraba al resto y mantiene `main` funcionando. En esta fase además es dueño de la automatización.

- **Tareas reales de Celery (semana 1 y 2).** En `cines/tasks.py`, una tarea `scrape_cartelera_task(cine, ciudad)` que solo llama a `scrape_and_save_cartelera`, y una tarea `limpiar_cartelera_vencida` que llama a `cleanup_expired_showtimes`. La lógica no se duplica: las tareas son envoltorios del servicio que ya existe.
- **Celery Beat y horarios.** Definir `CELERY_BEAT_SCHEDULE` en `base.py` con dos franjas diarias (por ejemplo, 6:00 a. m. y 6:00 p. m., hora de Bogotá) y la lista de pares cine–ciudad activos leída desde la configuración. Cada cine arranca unos minutos después del anterior para que las solicitudes queden espaciadas. Los horarios exactos se acuerdan con la Persona 3.
- **Resiliencia.** Reintentos con espera creciente ante `ScraperError` (máximo 3), límites de tiempo por tarea, y un candado en Redis para que no corran dos ejecuciones del mismo cine a la vez. Si un cine falla, los demás siguen.
- **Registro de ejecuciones.** Crear el modelo `EjecucionScraping` en `cines` (cine, ciudad, inicio, fin, estado, conteos del `CarteleraScrapeResult` y mensaje de error) con admin de solo lectura. Sirve para saber cuándo corrió cada scraper y por qué falló. Es una migración nueva en `cines`: se avisa a la Persona 3 antes de subirla porque ella también puede tocar esa app.
- **Worker y Beat en local.** Dejar resuelto cómo se levantan el worker y Beat en el equipo (el reconocimiento usa PowerShell, así que seguramente varios trabajan en Windows: Celery 5 necesita `--pool=solo` allí). Las pruebas de tareas corren en modo síncrono (`task_always_eager`), sin depender de Redis en la integración continua.
- **Revisión y merge** de todos los PRs de la fase, resolviendo conflictos de migraciones entre `cines`, `media` y `social`.
- **Decidir dónde se despliega** el proyecto (backend, worker, Beat, PostgreSQL y Redis) antes del 13 de diciembre, para que la Fase 6 no empiece sin entorno.

Terminado cuando: las tareas de scraping y limpieza corren solas según el horario, cada ejecución queda registrada en el admin, un fallo en un cine no detiene a los demás y cualquier integrante puede levantar worker y Beat en su máquina.

6. Persona 2 — Catálogo y scraper de Cineland

El catálogo ya está cerrado, así que la Persona 2 asume un scraper para repartir la carga. El contrato común lo publica la Persona 3 al final de la semana 1; hasta entonces la Persona 2 trabaja en el catálogo.

- **Corregir la prueba que nunca corre (semana 1).** Sacar `test_no_pisa_estado_ni_fecha_de_titulos_existentes` de dentro de la primera prueba de `media/tests/test_seed_data.py`, ejecutarla y arreglar lo que falle. Limpiar también los imports duplicados de `scrape_cartelera.py` y de `test_scrape_command.py`.
- **Alta ágil de fichas desde los títulos pendientes (semana 1 y 2).** En el admin de `TituloPendiente`, agregar la acción “Crear ficha borrador en el catálogo”: crea una `Media` de

tipo película con el título normalizado, estado `en_cartelera` y los demás campos vacíos, y deja el título pendiente como resuelto y asociado. Es una acción manual de un administrador, no una creación automática, así que se respeta la regla de no inventar fichas. Se acuerda con la Persona 3 para que el matching la reconozca en la siguiente ejecución.

- **Completar las fichas de lo que está en cartelera.** Cargar en `seed_data` (o en un comando aparte, igual de idempotente) las películas que hoy están en los cines de Valledupar, con sinopsis, póster, tráiler, géneros y reparto. Es lo que permite que el home de la Fase 5 muestre cartelera real y no una lista de pendientes.
- **Scraper de Cineland (semanas 2 y 3).** Crear `cines/scrapers/cineland.py` con el mismo patrón de Cinemark: una función que consulta y funciones de análisis que se prueban sin red. Datos del reconocimiento: `POST https://cineland.website/api_point_back.php` con `{"fechacon1", "fechacon2", "codteatro": "0051"}`; la respuesta es JSON aunque llega con content type `text/html`, así que se decodifica a mano. Cada película trae `fechas[]` → `atributos[]` → `salas[]` → `funciones_y_tarifas[]`; el formato sale de `atributos_pelicula`, la sala de `nombre_sala` y el título de `nombre_pelicula_original`. Consultar un rango corto de días (por ejemplo, 7) con una petición por día y pausas, no un rango enorme.
- **Opcional, si sobra tiempo:** un comando que complete sinopsis, póster y reparto de una ficha borrador desde TMDB. Requiere una clave en `.env` y mostrar la atribución que pide TMDB. Solo se hace si lo anterior ya está terminado.
- **Revisar** en PR el scraper de Cinépolis y el de Royal Films de la Persona 3.
- **Pruebas** de la acción de admin, del scraper de Cineland (con su respuesta de ejemplo) y de la prueba corregida.

Terminado cuando: el scraper de Cineland carga las funciones de Valledupar con un solo comando, un administrador puede convertir un título pendiente en una ficha desde el admin con un clic, y las películas que hoy están en cartelera existen en el catálogo con póster y sinopsis.

7. Persona 3 — Cartelera: contrato común, Cinépolis y Royal Films

- **Contrato común de los scrapers (semana 1, primero).** Mover a `cines/services.py` el formato que hoy es de Cinemark (`CarteleraData` con `funciones` y `proximos_estrenos`) y reemplazar `_get_scraper` y `CINEMA_WEBSITES` por un único registro (`SCRAPERS`) donde cada cine declara su función de scraping y su sitio web. Cinemark se adapta sin cambiar su comportamiento: sus pruebas siguen pasando. Debe estar publicado el viernes 20 de noviembre para que la Persona 2 empiece Cineland sobre él.
- **Ciudad canónica (semana 1).** Definir una configuración con la ciudad canónica (`valledupar`) y la clave que usa cada proveedor: `valledupar-colombia` en Cinépolis, `1006` en Royal Films, `0051` en Cineland. `Cine.ciudad` se guarda siempre con la ciudad canónica, para que `?ciudad=valledupar` devuelva funciones de todos los cines. Cinemark se mantiene configurado, pero solo se programa para una ciudad si de verdad devuelve funciones.
- **Scraper de Cinépolis (semanas 1 y 2).** `POST https://cinepolis.com.co/Cartelera.aspx/GetNowPlayingByCity` con

`{"claveCiudad":"valledupar-colombia","esVIP":false}`. La respuesta viene en el objeto `d`, con la ruta `Cinemas[]` → `Dates[]` → `Movies[]` → `Formats[]` → `Showtimes[]`. El formato se arma con `Name` y `Language` (por ejemplo, `DOB` + `DOBLADA`) y se guardan `ShowtimeId` y `CinemaId` como identificadores de origen. Es una sola llamada por ciudad. Antes de escribir el analizador, repetir la petición una vez y guardar la respuesta recortada como respuesta de ejemplo.

- **Scraper de Royal Films (semanas 2 y 3).** `GET /api/billboard/city/1006` para las películas y `GET /api/movies/functions/{peliculaId}/city/1006` por cada una, con pausa entre llamadas. Ojo con la hora: llega como `1970-01-01T16:30:00.000Z`, pero el valor real es la hora local (16:30), así que **no se convierte de zona horaria**: se toma la hora y los minutos y se combinan con `funcion_fecha`. Cine, sala y formato salen de `multicine_nombre`, `sala_nombre` y `formato_nombre`.
- **Mejorar el matching de títulos.** Agregar un campo opcional `titulo_alternativo` al formato común (Cinépolis entrega `Title` y `OriginalTitle`) y hacer que `match_media_title` pruebe ambos. Revisar si los títulos de Cinépolis, Royal y Cineland traen sufijos nuevos (por ejemplo, `3D`, `DOB`, `SUB`) y agregarlos a `TITLE_QUALIFIERS` con su prueba.
- **Sincronizar funciones (semana 3).** Al terminar un scrape exitoso de un cine, eliminar las funciones futuras de ese cine que ya no aparecen en la fuente. Salvaguarda obligatoria: si la respuesta vino vacía o con muy pocas funciones frente a la ejecución anterior, no se borra nada y se registra una advertencia, porque eso suele indicar que el sitio cambió y no que el cine cerró. Con esto se eliminan las funciones fantasma y los duplicados por cambio de sala o formato.
- **Decidir si hace falta Playwright.** El roadmap lo menciona, pero los cuatro sitios reconocidos se leen con peticiones HTTP directas. Se deja asentado que Playwright solo se agrega si algún sitio llega a exigir un navegador.
- **Endpoints de apoyo al home.** Agregar el filtro `?ciudad=` a `/api/cines/` (reutilizando `normalize_city`) y devolver `ultima_actualizacion` también en `/api/funciones/` y `/api/cines/<id>/funciones/`, para que el frontend muestre qué tan reciente es la información.
- **Revisar** en PR el scraper de Cineland de la Persona 2, y la configuración de Beat de la Persona 1 en lo que toca a cines y ciudades.
- **Pruebas** de cada scraper con su respuesta de ejemplo, del contrato común, del matching con título alternativo y de la sincronización (incluida la salvaguarda).

Terminado cuando: Cinépolis y Royal Films cargan las funciones reales de Valledupar con un solo comando cada uno, todos los scrapers hablan el mismo formato, una función retirada de la fuente desaparece de la base sin borrar nada por error, y `/api/funciones/?ciudad=valledupar` devuelve funciones de los tres cines con su fecha de actualización.

8. Persona 4 — Usuarios, capa social y base del frontend

Con el backend de usuarios cerrado, la Persona 4 adelanta la capa social para que la Fase 4 empiece con los modelos construidos, y deja el frontend preparado para la Fase 5.

- **Modelos de la capa social (semanas 1 a 3).** Implementar los modelos aprobados en la Fase 2: `Lista` (con sus elementos: favoritas, por ver y personalizadas), `Seguimiento` y `SugerenciaEdicion`, con migraciones, admin y pruebas. Las migraciones son de `social`; se avisa a la Persona 1 antes de subirlas. Los endpoints de `Seguimiento` y el flujo de aprobación de sugerencias quedan para la Fase 4.
- **Endpoints de listas (semanas 3 y 4).** Crear, ver, editar y borrar listas propias, agregar y quitar títulos, y ver las listas públicas de otro usuario, reutilizando `EsAutorOLecturaPublica`. Todos los errores usan el formato único de la API.
- **Limpiar el frontend actual (semana 1).** Leer la URL de la API desde una variable de entorno (`VITE_API_URL`) en lugar de la dirección fija de `services/api.js`; corregir los textos que dicen "mínimo 6 caracteres" cuando el código exige 8; quitar los botones de Google y Apple y el enlace "¿Olvidaste tu contraseña?", o marcarlos como "próximamente", para que no haya botones que no hacen nada.
- **Separar `App.jsx` en componentes (semana 2).** Hoy concentra el login, el registro, el perfil y el cambio de contraseña en un solo archivo muy largo. Dividirlo (por ejemplo, formulario de acceso, perfil y cambio de contraseña) sin cambiar el comportamiento, para que la Fase 5 no empiece sobre un archivo imposible de mantener.
- **Capa de servicios para el catálogo y la cartelera (semana 3).** Agregar a `services/api.js` las funciones que consumirá el frontend (`getMedia`, `getMediaDetalle`, `getFunciones`, `getCines`, `getPersona`) con el mismo manejo de errores y de refresco de token. No se construyen pantallas: es solo la capa que las alimentará.
- **Pasar los wireframes aprobados a variables de diseño (semana 4).** Colores, tipografía y espaciado en variables CSS (`index.css` ya tiene la base), de modo que la Fase 5 construya sobre una identidad ya decidida.
- **Pruebas** de los modelos nuevos, de los endpoints de listas y de sus permisos.

Terminado cuando: `Lista`, `Seguimiento` y `SugerenciaEdicion` existen con migraciones y admin, un usuario puede crear y gestionar sus listas por la API, el frontend no tiene botones muertos ni la URL de la API fija, y `services/api.js` tiene todas las llamadas que necesita la Fase 5.

9. Puntos de integración de la Fase 3

Punto	Quiénes	Qué acordar	Cuándo
Contrato común de scrapers	Persona 3 (publica), Personas 1 y 2	Forma de `CarteleraData`, el registro `SCRAPERS` y cómo se declara un cine nuevo. Cinemark adaptado sin cambios de comportamiento.	Semana 1, antes del 20 nov
Ciudad canónica	Personas 2 y 3	`Cine.ciudad` siempre con el slug `valledupar`; cada proveedor mapea su clave propia a esa ciudad.	Semana 1

Punto	Quiénes	Qué acordar	Cuándo
Tareas de Celery	Personas 1 y 3	`scrape_cartelera_task(cine, ciudad)` solo llama al servicio; un cine nuevo se programa agregándolo al registro y a la configuración.	Semana 2
Horarios y espaciado	Personas 1 y 3	Franjas del día, separación entre cines y cantidad máxima de ejecuciones diarias.	Semana 2
Migraciones nuevas	Personas 1, 3 y 4	`EjecucionScraping` (P1), posibles cambios en `cines` (P3) y modelos de `social` (P4): orden y nombres, avisando antes de subirlas.	Semanas 1 a 3
Títulos pendientes y fichas borrador	Personas 2 y 3	Cómo la acción del admin crea la `Media` y cómo el matching la reconoce en la siguiente ejecución.	Semana 2
Revisión cruzada de scrapers	Personas 2 y 3	La 2 revisa Cinépolis y Royal Films; la 3 revisa Cineland.	Semanas 2 y 3
Sincronización de funciones	Personas 1 y 3	Qué se registra en `EjecucionScraping` cuando la salvaguarda impide borrar funciones.	Semana 3
Entorno de despliegue	Persona 1 (con todos)	Dónde correrán backend, worker, Beat, PostgreSQL y Redis.	Antes del 13 dic

10. Criterio de cierre de la Fase 3

- Cinépolis, Royal Films y Cineland cargan funciones reales de Valledupar, cada uno con su propio comando y todos con el mismo formato común.
- La cartelera se actualiza sola, una o dos veces al día, con Celery Beat; cada ejecución queda registrada y un fallo en un cine no detiene a los demás.
- Las funciones retiradas de la fuente desaparecen y no hay duplicados por cambios de sala o formato.
- Las películas que hoy están en cartelera existen en el catálogo, y el admin permite dar de alta una ficha desde un título pendiente con un clic.
- `/api/funciones/?ciudad=valledupar` y `/api/cines/?ciudad=valledupar` devuelven datos reales con su fecha de última actualización.
- Los modelos de la capa social existen con migraciones y admin, y las listas funcionan por la API.
- El frontend no tiene botones muertos, usa la URL de la API desde el entorno y tiene lista la capa de servicios para la Fase 5.
- Todo está en `main`, con las pruebas pasando en la integración continua.

11. Documentación

La documentación en formato `.md` (README, guías, informes de reconocimiento y demás textos de apoyo) **no forma parte de las responsabilidades de esta fase**. Esos documentos se compartirán con el grupo por WhatsApp.